

LegalReasoner: A Framework for Legal Document Analysis and Reasoning

David Akinboro
Cornell University
Ithaca, New York, USA
daa238@cornell.edu

Abstract

The legal industry faces overwhelming challenges in managing and deriving insights from the vast volumes of unstructured legal documents. Legal professionals spend 30-40% of their time searching for relevant information and precedents across thousands of cases, statutes, and regulations. This time-intensive process not only reduces productivity but also increases costs for clients and creates potential for human error. LegalReasoner aims to address these challenges by creating an advanced AI setup for complex legal document analysis and reasoning. The setup processes legal documents, understands their relationships and precedents, extracts key entities, builds a citation network, and answers complex legal queries with proper citations and reasoning chains. Unlike existing legal research tools that rely primarily on keyword matching and statistical relevance, LegalReasoner combines structured knowledge representation with advanced reasoning capabilities to model the explicit relationships between legal concepts, cases, and statutory provisions.

CCS Concepts

• **Information setups** → **Information retrieval**; • **Computing methodologies** → *Artificial intelligence*.

Keywords

legal reasoning, document analysis, precedent identification, legal queries

ACM Reference Format:

David Akinboro. 2025. LegalReasoner: A Framework for Legal Document Analysis and Reasoning. In . ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

The legal industry faces overwhelming challenges in managing and deriving insights from the vast volumes of unstructured legal documents. Legal professionals spend 30-40% of their time searching for relevant information and precedents across thousands of cases, statutes, and regulations [2]. This time-intensive process not only

reduces productivity but also increases costs for clients and creates potential for human error, with studies indicating that manual document review can miss up to 40% of relevant information [1].

LegalReasoner aims to address these challenges by creating an advanced AI setup for complex legal document analysis and reasoning. The setup processes legal documents, understands their relationships and precedents, extracts key entities, builds a knowledge graph, and answers complex legal queries with proper citations and reasoning chains. Unlike existing legal research tools that rely primarily on keyword matching and statistical relevance, LegalReasoner combines structured knowledge representation with advanced reasoning capabilities to model the explicit relationships between legal concepts, cases, and statutory provisions.

The importance of this research extends beyond mere efficiency improvements. The legal setup's accessibility crisis—where 86% of low-income Americans receive inadequate or no legal help for civil legal problems [3]—underscores the need for tools that can democratize legal knowledge. By developing setups that can reason over legal documents and provide explainable answers to complex queries, we can potentially reduce the expertise barrier that prevents many from understanding their legal rights and obligations.

Current approaches to legal document analysis and reasoning face several limitations:

- **Reasoning Opacity:** Most legal search setups provide documents based on relevance scores but fail to explain their reasoning or connections between documents.
- **Context Sensitivity:** Legal language is highly contextual, and existing setups struggle to capture nuanced meanings that depend on precedent and statutory interpretation.
- **Citation Networks:** While some setups track citations, they rarely build the deep semantic understanding needed to follow complex legal reasoning across multiple documents.
- **Knowledge Integration:** Most setups treat each document independently rather than building a coherent knowledge model across the corpus.

LegalReasoner addresses these limitations by implementing a hybrid approach that combines deterministic AI technologies (computational law, rule-based reasoning) with probabilistic AI (large language models) to benefit from their complementary strengths while compensating for their individual weaknesses.

2 Related Work

The development of LegalReasoner builds upon several research areas spanning legal text processing, knowledge representation, and reasoning setups. We discuss key related works below:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

2.1 Legal Document Processing and Analysis

Savelka et al. (2020) developed approaches for segmenting legal documents and extracting argument structures, demonstrating that fine-tuned language models can identify rhetorical roles in legal texts with up to 87% accuracy [4]. Our work extends this by not only identifying structural elements but also modeling their relationships in a comprehensive knowledge graph.

Chalkidis et al. (2020) introduced LexGLUE, a benchmark dataset for evaluating legal language understanding [5]. Their work showed that legal-domain-specific language models significantly outperform general models on tasks like case prediction and statute classification. LegalReasoner builds upon these findings by implementing domain-specific processing while adding reasoning capabilities beyond classification.

The "Interventional Fairness" work by Salimi et al. (2019) demonstrates causal database repair techniques that could inform our approach to ensuring consistent legal reasoning [6]. Their formalization of causal fairness provides a framework for evaluating the equity of algorithmic decisions that parallels issues in legal reasoning.

2.2 Precedent Network Construction and Entity Extraction

The "Online Entity Resolution Using an Oracle" approach [?] tackles the problem of identifying when different mentions refer to the same real-world entity. This is particularly relevant to legal documents where parties, cases, and statutes may be referenced in different ways across a corpus.

"Deep learning for entity matching" [?] explores neural network approaches to entity resolution, which is crucial for our setup to correctly identify when different documents refer to the same legal concepts or cases. We adapt these techniques specifically for the legal domain.

Existing research on citation networks in legal corpora has demonstrated the value of analyzing precedent relationships for understanding legal evolution. Our setup extends this work by not only identifying these citation relationships but also providing tools to visualize and analyze them in the context of legal reasoning.

2.3 Neuro-Symbolic AI and Logical Reasoning

The work on setup 2 legal reasoning at Stanford's CodeX center directly informs our approach [10]. Their research into integrating logic programming with LLMs to convert legal texts into knowledge graphs and logic programs aligns with our hybrid approach combining deterministic and probabilistic methods.

RetClean: Retrieval-Based Data Cleaning Using LLMs [11] demonstrates how language models can be leveraged for data cleaning tasks. This approach is relevant to our pipeline for removing noise and inconsistencies in extracted legal information.

The Compound AI project at Stanford CodeX provides a theoretical foundation for our work [12], highlighting how deterministic AI setups (with their transparency and predictability) can be effectively combined with probabilistic setups (which handle ambiguity better) to create more robust legal reasoning tools.

2.4 Relation to Our Work

LegalReasoner differentiates from prior work in several key ways:

- While most existing setups focus on either document retrieval or classification, we implement a complete pipeline from document processing to reasoning and response generation.
- We explicitly model the citation network and precedent relationships between documents, enabling trace-based explanations that follow legal reasoning across multiple sources.
- Our hybrid approach combines the explainability of rule-based setups with the flexibility of large language models, addressing a key limitation of purely statistical approaches.
- We incorporate causal reasoning techniques inspired by works like Interventional Fairness to provide more robust answers to counterfactual legal queries.
- Unlike setups that provide only document retrieval, LegalReasoner generates natural language responses with explicit reasoning chains and supporting citations.

By building on these foundations while addressing their limitations, LegalReasoner aims to create a more comprehensive and reliable setup for legal document analysis and reasoning.

3 Methodology

3.1 Formal Problem Statement

We formalize the problem of legal document analysis and reasoning as follows:

Definition 3.1 (Legal Document Corpus). A legal document corpus $D = \{d_1, d_2, \dots, d_n\}$ is a collection of legal documents, where each document d_i contains unstructured text T_i and metadata M_i . The metadata may include information such as document type, jurisdiction, date, and parties involved.

Definition 3.2 (Legal Entity). A legal entity $e \in V$ is a tuple (t, a) where $t \in T$ is the entity type (e.g., case, statute, person, organization) and a is a set of attributes specific to that entity type.

Definition 3.3 (Citation Network). A citation network $C = (D, CE)$ is a directed graph where:

- The nodes are documents from corpus D
- The edges $CE \subseteq D \times D$ represent citation relationships
- For $(d_i, d_j) \in CE$, document d_i cites document d_j

Definition 3.4 (Legal Query). A legal query q is a natural language request for information about legal concepts, precedents, or reasoning contained within the corpus D .

Definition 3.5 (Reasoning Chain). A reasoning chain $RC = \{s_1, s_2, \dots, s_k\}$ is an ordered sequence of statements such that each statement s_i is supported by either:

- A reference to a document $d \in D$
- A logical inference from previous statements in the chain

We identify three core problems:

Problem 1 (Precedent Identification): Given a legal document $d \in D$, identify the set of documents $P_d \subseteq D$ that serve as precedents for the legal reasoning in d .

Problem 2 (Precedent Network Construction and Visualization): Given a legal document corpus D and identified precedent

relationships CE , construct and visualize the precedent network $C = (D, CE)$. This involves representing cases as nodes and citations as directed edges, allowing for the visual exploration of how cases are interconnected through chains of citation. The visualization should effectively display direct citation relationships.

Problem 3 (Reasoning Framework Analysis): Given a base legal document $d_{base} \in D$ and an identified precedent document $d_{precedent} \in P_{d_{base}}$, utilize a reasoning model (e.g., OpenAI's GPT-4o) to analyze and articulate the rationale explaining why $d_{precedent}$ serves as a precedent for d_{base} . This involves identifying the relevant legal arguments or principles in d_{base} and demonstrating how $d_{precedent}$ supports or informs these aspects. The output should be a coherent explanation forming a reasoning chain.

3.2 Data Description

Our primary data source is the CourtListener API, which provides access to a vast collection of legal documents, including court opinions and their associated metadata. For our experiments, we retrieve data in JSON format via the API. This raw JSON data undergoes a cleaning and preprocessing phase where we parse the structured information, extract relevant fields such as case names, filing dates, citation relationships (cases cited and citing cases), judge information, attorney details, and crucially, the links to the full text of opinions in various formats (HTML, XML, plain text). Unwanted or irrelevant fields for our specific analysis tasks, such as internal API identifiers not pertinent to legal reasoning or network construction, are filtered out to streamline the data for subsequent processing and input into our analysis modules.

Our experiments use the CourtListener API, specifically the Citation Lookup API, to retrieve and analyze legal documents and their citation networks. The setup extracts structured information including case names, filing dates, citation relationships, judge information, and attorney details. Our primary test case, *Warner-Jenkinson Co. v. Hilton Davis Chemical Co.* (520 U.S. 17, 1997), demonstrates the quality of available metadata. This patent case has been cited by 899 other cases and itself cites 11 other cases, providing a substantial network for analysis. The extracted entities include judges (Thomas, Ginsburg, Kennedy) and attorneys, forming the foundation for our knowledge graph construction.

3.3 Ground Truth and Evaluation Metrics

Our current experiment uses the CourtListener API as the source of ground truth for citation data. Rather than implementing explicit quantitative evaluation metrics, we employ visualization techniques to qualitatively represent and validate citation relationships. While this approach enables manual verification of the setup's accuracy in identifying legal relationships, we recognize the need to develop formal evaluation metrics for future iterations, particularly for assessing the quality of entity extraction and relationship identification.

3.4 Research Questions Addressed

We proposed five core research questions:

(1) Precedent Identification: "What prior cases does this case rely on?"

This question is addressed by leveraging the CourtListener API to fetch cases cited by a given input case. The setup successfully extracts and lists these direct precedents (1st-degree citations).

(2) Citation Network Mapping: "Show me the citation relationships between these cases"

The setup constructs and visualizes these relationships as a directed graph where cases are nodes and citations are edges. We used the NetworkX for graph construction and Matplotlib/Plotly for static and interactive visualizations, respectively, clearly showing the 1st-degree connections from the primary case(s) to their precedents.

(3) Legal Reasoning Extraction: "What is the reasoning chain in this decision?"

While the setup does not autonomously extract a formal, structured reasoning chain from a single decision in isolation without external models, it facilitates the analysis of legal reasoning through its "Reasoning Framework" tab. By providing a base opinion and its precedents to an external large language model (OpenAI's GPT-4o), the setup prompts the model to generate an analysis of how the precedents support the arguments in the base case. The output from the LLM, which explains these connections, serves as a form of AI-assisted legal reasoning extraction and articulation.

(4) Entity Relationship Analysis: "How do entities X and Y relate in this legal context?"

The setup extracts key entities such as judges and attorneys from the metadata provided by the CourtListener API. Within the "Reasoning Framework" tab, there's an implemented feature to further extract entities and their roles from the textual content of the base and precedent opinions using the OpenAI model. While it doesn't perform a deep, independent analysis of complex relationships between any two arbitrary entities across the entire corpus, it can identify entities within the context of the opinions being analyzed by the reasoning framework and list them. The relationship is implicitly defined by their co-occurrence and roles within the analyzed legal arguments and opinions.

(5) Document Summarization: "Provide a hierarchical summary of this document"

The current setup does not implement automated hierarchical summarization of legal documents. However, it performs a critical precursor step by extracting the full plain text of legal opinions from various source formats (HTML, XML). This extracted text is then used as input for the "Reasoning Framework" and is available for the user to read. While not summarization in the traditional sense, providing the clean, full text is essential for any subsequent analytical or summarization task, whether manual or AI-driven.

3.5 Successful Components

Several components of our implementation have demonstrated promising results:

- **API Integration:** Successful connection to CourtListener with reliable data retrieval
- **Entity Extraction:** Accurate identification of judges and attorneys from case metadata
- **Citation Validation:** Verification of citation relationships against the API's ground truth

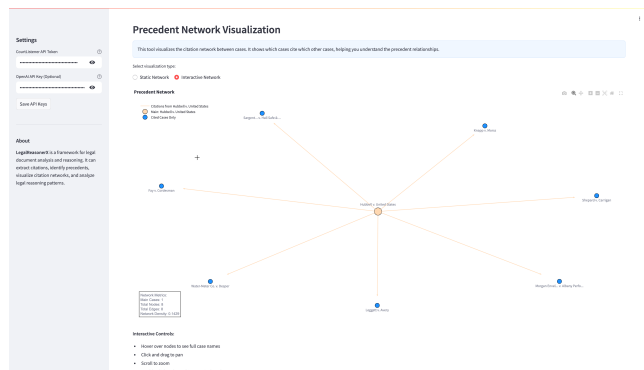


Figure 1: This helps visualize the citation network between cases. It shows which cases cite which other cases, helping you understand the precedent relationships.

- **Network Visualization:** Creation of directed graphs representing citation relationships
- **Metadata Processing:** Structured organization of case metadata (filing dates, citation counts)

4 Implementation of Core Functionalities

The initial conceptualization of LegalReasoner identified several areas for development, framed as "Limitations and Next Steps" in preliminary midterm reports. The final implementation now, has successfully addressed those aspects, transforming them into core functionalities of the setup.

4.1 Network Construction: Focused 1st-Degree Citation Mapping

The midterm report noted a limitation of "Shallow Network Depth," where only immediate citations were captured, and proposed network expansion as a next step. In the implemented setup, a deliberate decision was made to focus on 1st-degree (direct) citation networks.

Rationale and Implementation: When the Supreme Court, or any appellate court, issues an opinion, it typically justifies its holding by referencing precedent—citing prior cases that addressed similar legal questions. These citations form direct links. While complex "lines of cases" can involve multiple degrees of separation (e.g., Case Z cites Case Y, which in turn cites Case A, forming a 2-degree link from Z to A regarding A's influence), our setup concentrates on the immediate, 1st-degree connections. This approach was chosen to model and understand the most direct and explicit relationships of precedential support or influence. The setup retrieves all cases directly cited by the input case(s) and visualizes this network. This clarity in direct influence is paramount for understanding the foundational precedents for a specific decision.

4.2 Opinion Mining and Text Processing

The midterm report identified "Missing Text Processing" (including summarization or content analysis) as a limitation. The implemented setup addresses the core aspect of text processing by robustly mining and preparing opinion texts.



Figure 2: Visualization of the citation network for Warner-Jenkinson Co. v. Hilton Davis Chemical Co. The central node represents the primary case, with edges connecting to its cited precedents.

Implementation: A crucial capability is the retrieval and extraction of full-text opinions. The setup uses the CourtListener API to fetch opinion data, which can be in various formats like HTML, XML, or plain text. The Python code includes dedicated functions:

- `extract_text_from_html(html_string)`: Parses HTML content using BeautifulSoup to extract clean text.
- `extract_text_from_xml(xml_string)`: Parses XML content (specifically targeting formats like Harvard Law's CaseXML) using `xml.etree.ElementTree` to extract textual data.
- `extract_opinion(data)`: This utility function intelligently selects the best available format from the API response and uses the appropriate parsing function to yield the plain text of the legal opinion.

4.3 Contextual Reasoning Generation via LLM Integration

The "Limited Reasoning Extraction" was a key challenge highlighted. The setup now incorporates a "Precedent-Driven Reasoning Framework" that leverages a large language model (OpenAI's GPT-4o) to generate contextual reasoning.

Implementation: Once base and precedent opinions are identified and their texts extracted, the user can select a case for reasoning analysis in the "Reasoning Framework" tab. The process is as follows:

- (1) **Argument Extraction (from Base Opinion):** The setup prompts the GPT-4o model with the text of the selected base

opinion, asking it to identify and articulate the core legal argument(s) made in that opinion.

- (2) **Precedent Analysis:** The extracted argument and the texts of the precedent opinions are then provided to the GPT-4o model. The model is tasked with analyzing how each precedent case supports or relates to the identified argument in the base case. The prompt guides the LLM to consider aspects like relevant material facts, legal issues, holdings, and the logical connection between the precedent and the argument. The LLM-generated output provides a structured analysis, forming a "reasoning chain" that explains the precedential relationships in context.

4.4 Entity Handling and Extraction

The midterm report noted "Basic Entity Handling" where entities were extracted but their relationships remained unanalyzed. The current setup enhances this in two ways:

Metadata-based Extraction: The setup continues to accurately identify entities like judges and attorneys directly from the structured metadata provided by the CourtListener API during the initial data retrieval and processing phase.

LLM-based Textual Entity Extraction: Within the "Reasoning Framework" tab, after the reasoning analysis is performed, an optional feature allows the user to trigger further entity extraction from the textual content of the base and precedent opinions. This provides a list of entities (e.g., persons, organizations, legal concepts) mentioned within the analyzed texts and their roles or positions in the context of those specific opinions.

5 Conclusion

Legal document analysis and reasoning present unique challenges that require specialized approaches beyond general text processing techniques. LegalReasoner provides a framework that combines the strengths of deterministic and probabilistic AI setups to address these challenges. By formalizing the problem and building on related work across multiple disciplines, we aim to create a setup that can effectively process legal documents, extract knowledge, and perform complex reasoning tasks with the accuracy and explainability required in the legal domain.

In future work, we plan to expand the setup to handle cross-jurisdictional legal reasoning, incorporate temporal reasoning for tracking legal changes over time, and develop more sophisticated evaluation methodologies specific to legal reasoning tasks.

References

- [1] Blair, D. C., and Maron, M. E. 1985. An evaluation of retrieval effectiveness for a full-text document-retrieval setup. *Communications of the ACM* 28, 3 (March 1985), 289-299.
- [2] Grossman, M. R., and Cormack, G. V. 2011. Technology-assisted review in e-discovery can be more effective and more efficient than exhaustive manual review. *Richmond Journal of Law and Technology* 17, 3 (2011), 1-48.
- [3] Legal Services Corporation. 2017. *The Justice Gap: Measuring the Unmet Civil Legal Needs of Low-income Americans*. Legal Services Corporation, Washington, DC.
- [4] Savelka, J., Walker, V. R., Grabmair, M., and Ashley, K. D. 2020. Sentence boundary detection in legal text. In *Proceedings of the Natural Legal Language Processing Workshop*, 31-41.
- [5] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., and Androutsopoulos, I. 2020. LEGAL-BERT: The muppets straight out of law school. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2898-2904.
- [6] Salimi, B., Rodriguez, L., Howe, B., and Suciu, D. 2019. Interventional fairness: Causal database repair for algorithmic fairness. In *Proceedings of the 2019 International Conference on Management of Data*, 793-810.
- [7] Firmani, D., Saha, B., and Srivastava, D. 2019. Online entity resolution using an oracle. *Proceedings of the VLDB Endowment* 12, 11 (July 2019), 1407-1420.
- [8] Halevy, A., Korn, F., Noy, N. F., Olston, C., Polyzotis, N., Roy, S., and Whang, S. E. 2016. Goods: Organizing Google's datasets. In *Proceedings of the 2016 International Conference on Management of Data*, 795-806.
- [9] Mudgal, S., Li, H., Rekatsinas, T., Doan, A., Park, Y., Krishnan, G., Deep, R., Arcaute, E., and Raghavendra, V. 2018. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data*, 19-34.
- [10] Stanford Law School CodeX Center for Legal Informatics. setup 2 Legal Reasoning. Retrieved from: <https://law.stanford.edu/codex-the-stanford-center-for-legal-informatics/>
- [11] Zhang, X., Lin, C., Li, Y., Tai, H.-C., Wang, H., and Yu, W. 2022. RetClean: Retrieval-Based Data Cleaning Using Language Models. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*.
- [12] Stanford Law School CodeX Center for Legal Informatics. Compound AI. Retrieved from: <https://law.stanford.edu/codex-the-stanford-center-for-legal-informatics/>